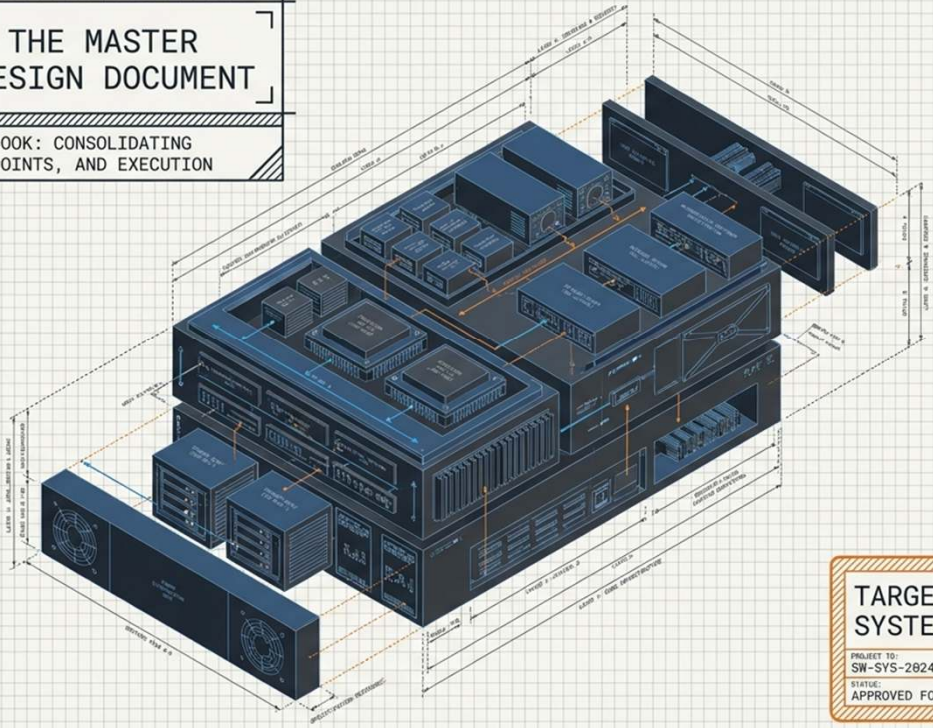
DELIVERING THE MASTER SOFTWARE DESIGN DOCUMENT

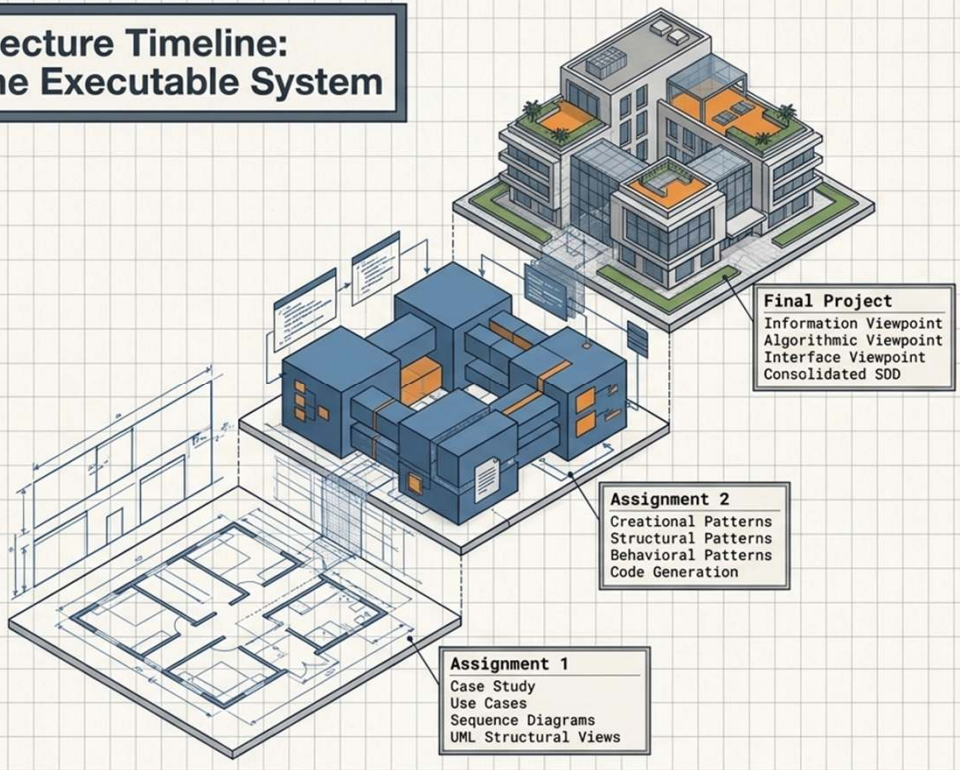
FINAL PROJECT PLAYBOOK: CONSOLIDATING ARCHITECTURE, VIEWPOINTS, AND EXECUTION



TARGET: FULL SYSTEM HANDOVER

PROJECT TO: SW-SYS-2824-v4	DATE: 26 OCT 2824
STATUS: APPROVED FOR RELEASE	REVISION: 4.8

The Architecture Timeline: Building the Executable System



Final Project
Information Viewpoint
Algorithmic Viewpoint
Interface Viewpoint
Consolidated SDD

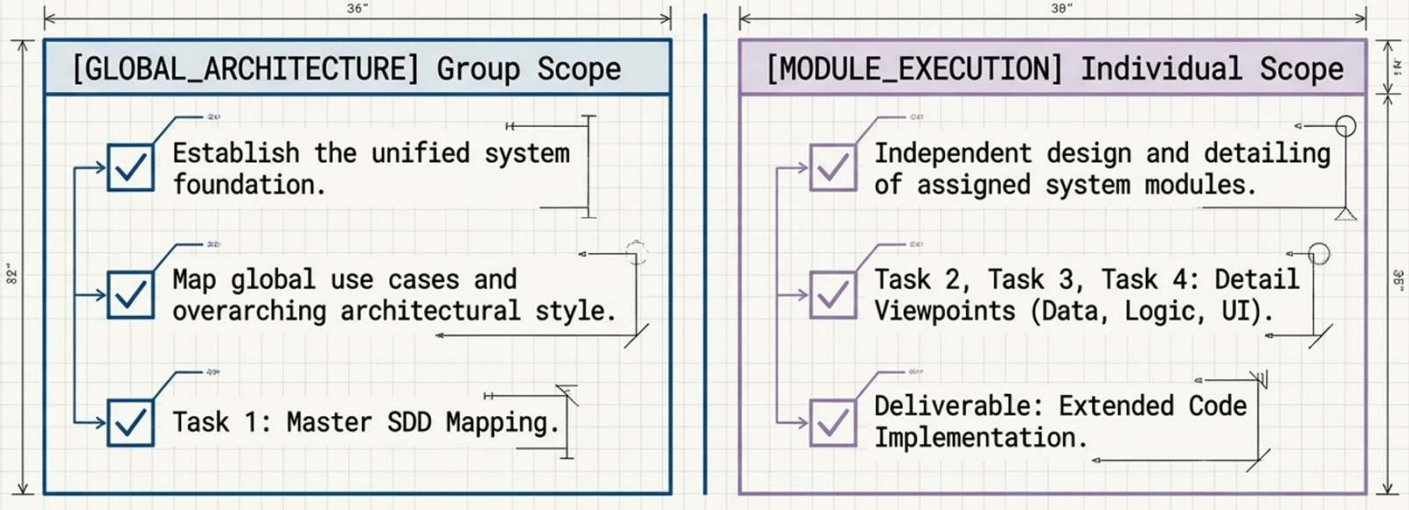
Assignment 2
Creational Patterns
Structural Patterns
Behavioral Patterns
Code Generation

Assignment 1
Case Study
Use Cases
Sequence Diagrams
UML Structural Views

The Final Project does not discard your previous work; it encapsulates it.

You will map your Assignment 1 blueprint and Assignment 2 engine into a single, cohesive, client-ready Software Design Document.

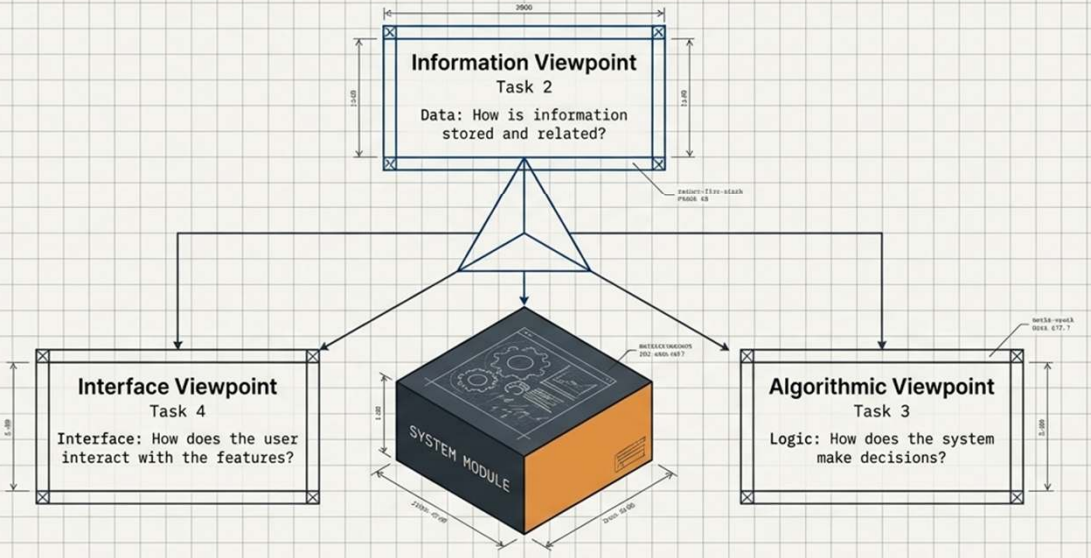
Defining the Scope Boundaries: Global vs. Module Execution



CRITICAL RULE: One module can contain multiple use cases. Individual work MUST be explicitly labeled in the final report (e.g., 2.1 Sequence for Module 1 - Member 1).

Detailing the Individual Modules: The Three Viewpoints

For every module you own, you must construct three specialized viewpoints to bridge the gap between high-level architecture and the end-user experience.



Task 2: The Information Viewpoint

Mapping the data entities required to execute your specific module functionalities.

Requirement

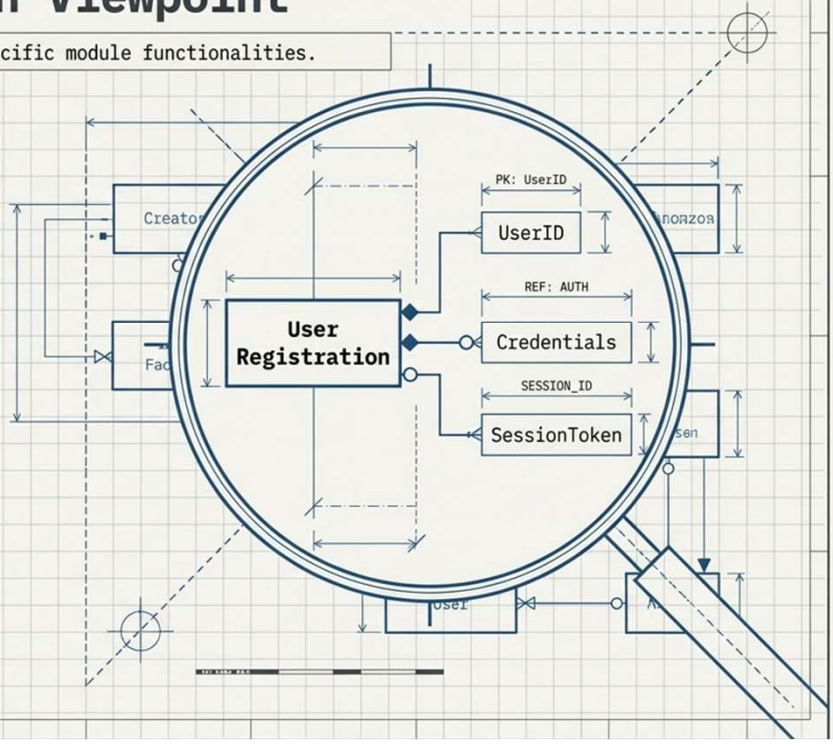
Create comprehensive Entity Relationship Diagrams (ERDs).

Focus Areas

Must explicitly cover core features within your module (e.g., User Registration, Order Processing).

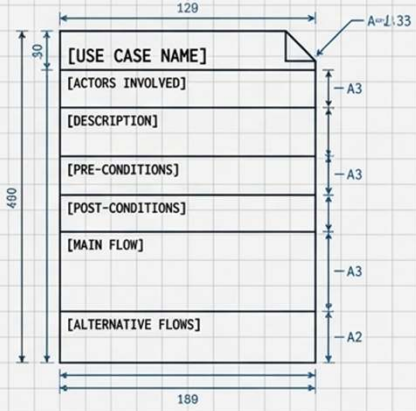
Alignment Check

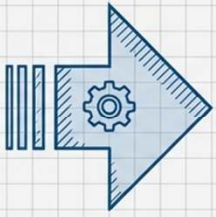
The entities defined here must mathematically align with the classes established in your Assignment 1 UML diagrams.

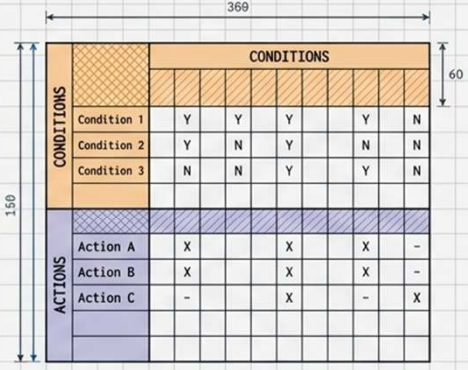


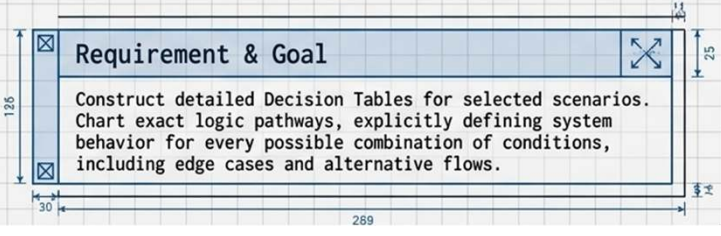
Task 3: The Algorithmic Viewpoint

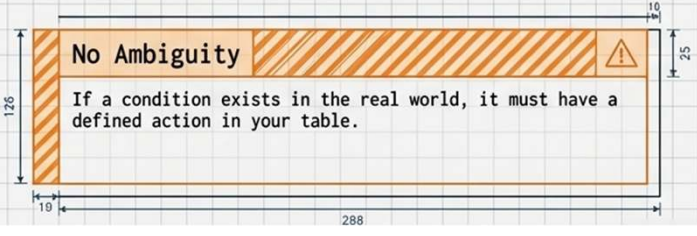
Translating narrative use cases into exact algorithmic logic pathways.











Task 4: The Interface Viewpoint

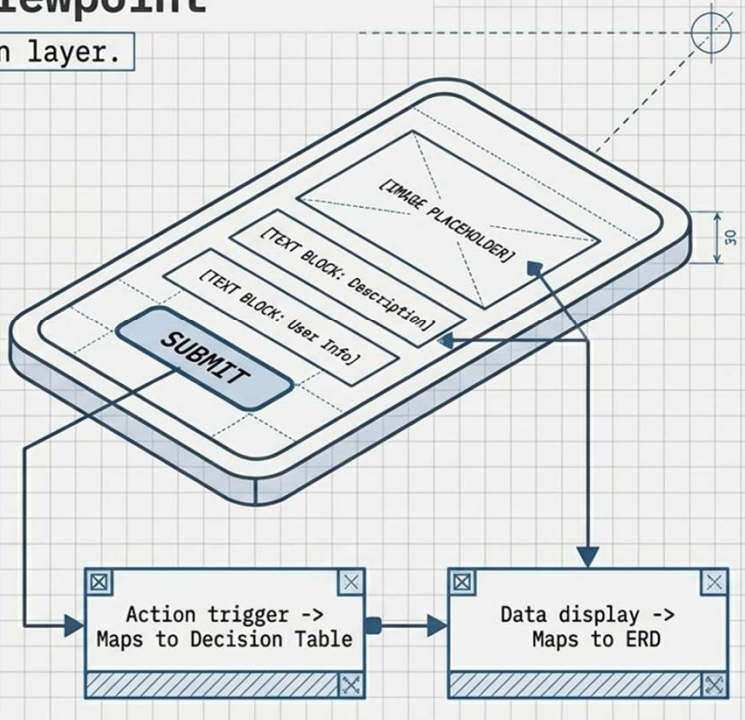
Designing the human-computer interaction layer.

Requirement

Develop UI Wireframe designs for each of your module's core features.

Integration Constraint

These are not just graphical mockups. Every button, form, and data display in the wireframe must trigger an action defined in your Decision Tables and interact with data mapped in your ERDs.



Deliverable: Extended Code Implementation

Your architecture is only as good as the code that executes it.

The Requirement

Expand upon the concrete code samples generated in Assignment 2.

The Pattern Triad

Your code must continue to utilize standardized design patterns (Creational, Structural, Behavioral) established previously.

Traceability

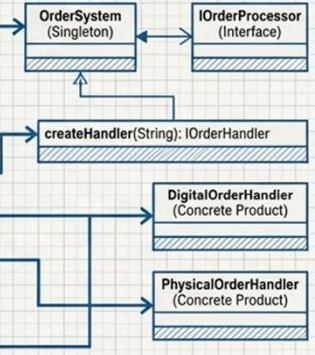
Syntax highlighting and heavy commenting are mandatory. Lines of code must explicitly map back to your architecture diagrams and your new Viewpoints.

```
public class OrderSystem implements IOrderProcessor {
   // SINGLETON: Ensures only one instance manages order processing across the system.
   private static OrderSystem instance;
   // PRIVATE CONSTRUCTOR: Prevents direct instantiation from outside.
   private OrderSystem() {
      System.out.println("OrderSystem Initialized.");
   }

   // FACTORY METHOD: Creates specific order handlers based on order type.
   @Override
   public IOrderHandler createHandler(String orderType) {
      if ("DIGITAL".equalsIgnoreCase(orderType)) {
         // Returns concrete handler for digital orders, mapping to DigitalOrderHandler
         return new DigitalOrderHandler();
      } else if ("PHYSICAL".equalsIgnoreCase(orderType)) {
         // Returns concrete handler for physical orders, mapping to PhysicalOrderHandle
         return new PhysicalOrderHandler();
      }
      return null; // Or throw exception for unknown types.
   }

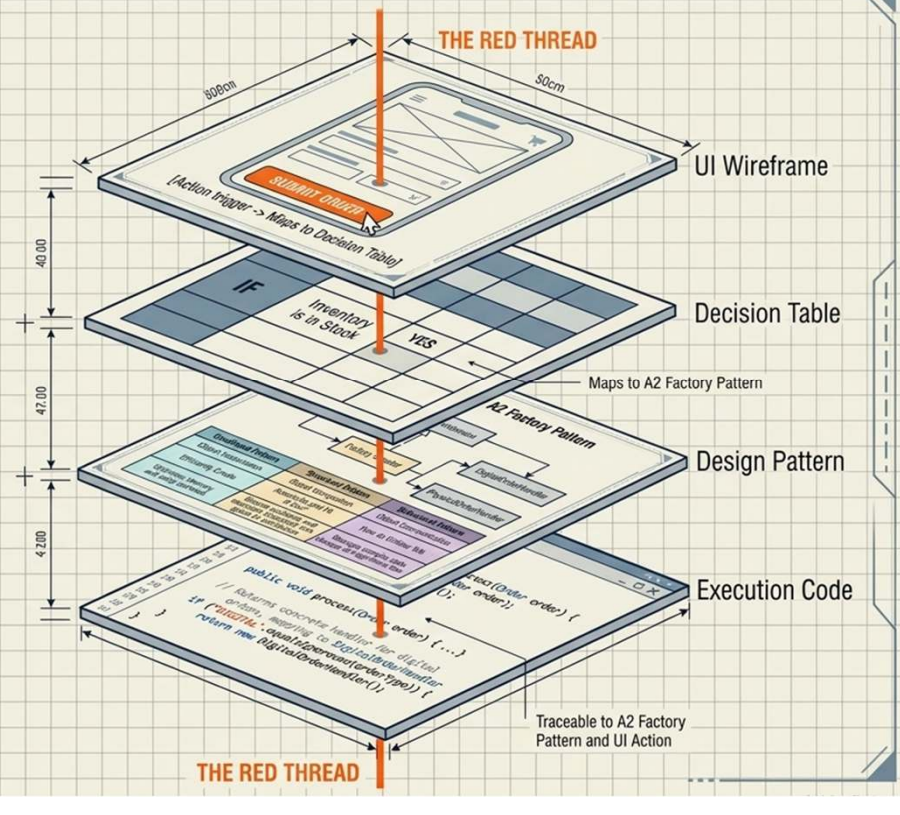
   // GLOBAL ACCESS POINT: Provides the single instance of OrderSystem.
   public static synchronized OrderSystem getInstance() {
      // LAZY INITIALIZATION: Created only when first needed.
      if (instance == null) {
         instance = new OrderSystem();
      }
      return instance;
   }

   // Defines the interface for all order handlers.
   // Maps to the IOrderHandler interface in the UML diagram.
   public interface IOrderHandler {
      void process(Order order);
   }
}
```

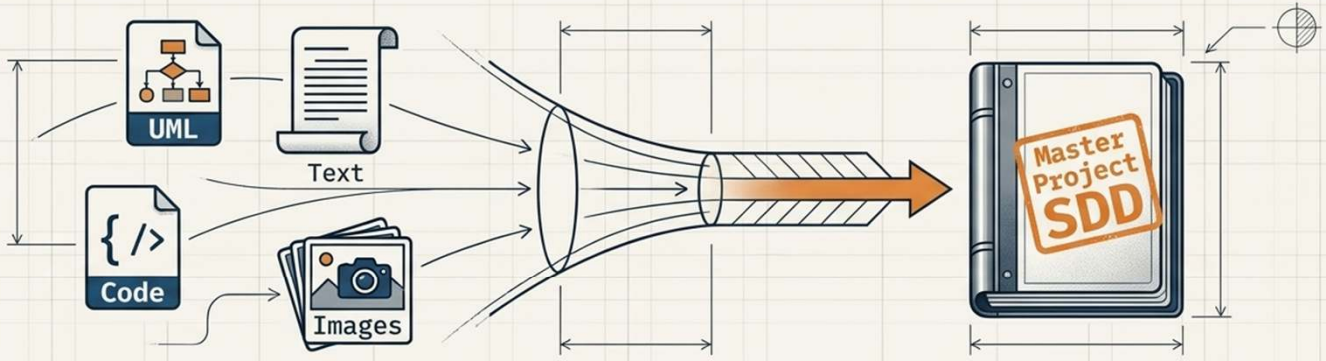


The Traceability Stack: Proving Architectural Coherence

The Final Project demands seamless, mathematical alignment from the user's screen down to the compiler. If an attribute exists in the UI, it must exist in the logic, the pattern, and the code.



Delivery Logistics: The Consolidated Report



Format

Final submission requires a single, consolidated report containing all diagrams, code snippets, and supporting documentation.

No Fragmentation

Do not submit separate folders of images or detached code files. Everything must live within the standardized SDD narrative.

Visual Standards

All diagrams must be clear, legible, and properly labeled.

NotebookLM

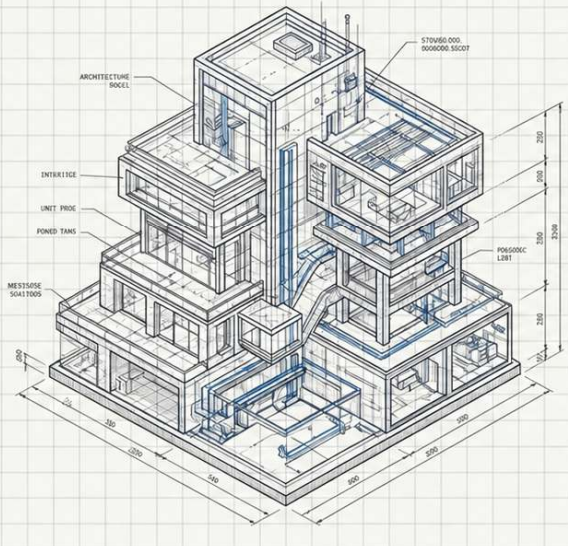
The Final QA Flight Check

Run this diagnostic before compiling your final SDD:

FINAL QA CHECKLIST

[ ]	Is individual work (Tasks 2, 3, 4, 6) explicitly labeled with the assigned member's name and module?
[ ]	Are all explanations concise and directly relevant to the core challenge?
[ ]	Is the global architectural style adequately justified against non-functional requirements?
[ ]	Does every code sample utilize required syntax highlighting and omit unnecessary boilerplate?

System Initialization: Awaiting Final Deployment



Final Deadline: Friday, 3rd July 2026

End of Lecture Semester

Format

A single, consolidated report containing all diagrams of future th, architecture and architecture.

Deadline

Friday, 3rd July 2026

Consolidate your blueprints. Implement your patterns. Deliver the architecture.